A Size-Invariant Deep Reinforcement Learning Policy for the Interval Job Shop Scheduling Problem

Hernán Díaz Rodríguez[1*]

[1*]Department of Computing, University of Oviedo, Gijón, Spain.

Corresponding author(s). E-mail(s): diazhernan@uniovi.es;

Abstract

Task durations on a shop floor are seldom known exactly when scheduling decisions must be made. The interval job shop scheduling problem (IJSP) models each processing time as a closed interval and evaluates schedules by interval arithmetic; published solvers search each instance separately with metaheuristics. This paper studies the complementary regime: a constructive policy, trained once by deep reinforcement learning, that dispatches operations one at a time and returns a schedule in milliseconds. The policy is size-invariant by construction: dimensionless features and a permutation-invariant encoder give policy and value heads a parameter count independent of the numbers of jobs and machines. Trained with proximal policy optimization on four **20×15** instances of the interval Taillard benchmark, the policy reaches **13.4%** mean relative error against crisp lower bounds on unseen instances of that size, against **≈46%** for the best dispatching rule on that class, and, evaluated zero-shot, beats it on every tested instance of all seven size classes; the same weights transfer to a second benchmark of classical instances. Against an evolved dispatching rule the ordering depends on the inference budget: pass for pass the rule wins, whereas at **1024** samples each the policy wins 46 of the 70 instances. Four ablations return negative results: self-attention does not improve on pooling; multi-size training does not overtake a single-size specialist even at three times its cost; and, most consequential for the design itself, removing the interval-width features changes nothing, and the crisp midpoint instances train just as good a policy.

Keywords: job shop scheduling, interval uncertainty, deep reinforcement learning, neural combinatorial optimization, size invariance, dispatching

1 Introduction

A production schedule is committed before the work it plans has been done, and the durations it assumes are estimates: tool wear, operator variability and material differences all move them. The job shop scheduling problem, NP-hard (Garey, Johnson, & Sethi, 1976) and industrially ubiquitous, is nonetheless posed in its textbook form as if every processing time were known exactly. Of the formalisms proposed for this uncertainty, stochastic and fuzzy models (González-Rodríguez, Vela, & Puente, 2010) demand distributions or membership functions from the decision maker; the *interval* model asks only for bounds. In the resulting interval JSP (IJSP), each duration is a closed interval, schedule evaluation propagates those intervals, and the makespan is itself an interval, compared through a ranking criterion (Díaz, Palacios, González-Rodríguez, & Vela, 2023a; Lei, 2011).

Makespan minimization in the IJSP is currently the territory of per-instance metaheuristic search: elitist artificial bee colony algorithms (Díaz et al., 2023a; Díaz, Palacios, González-Rodríguez, & Vela, 2023b) reach mean relative errors (RE, measured against crisp Taillard lower bounds) between roughly 6% and 16% depending on the size class. The price is structural: every new instance restarts the search from scratch, for seconds to minutes per run on dedicated hardware.

The deterministic scheduling literature has developed a complementary paradigm, *learning to dispatch*: a deep reinforcement learning (DRL) agent builds the schedule constructively, choosing one eligible operation at a time (Park, Chun, Kim, Kim, & Park, 2021; Tassel, Gebser, & Schekotihin, 2021; Zhang et al., 2020). The training cost is paid once; afterwards the policy prices any instance in milliseconds, which makes it attractive both as a standalone solver under tight time budgets and as an initializer for search. Yet this paradigm has not crossed over to interval durations: a recent survey of learning-based job shop scheduling (Xu, Mei, Zhang, & Zhang, 2025) lists no method addressing interval-valued processing times.

Two obstacles stand between standard JSP-DRL designs and the IJSP. The first is representational: uncertainty must be visible to the agent, which should treat an operation lasting $[10, 11]$ differently from one lasting $[5, 16]$ even though both share the same midpoint. The second is architectural: common designs tie network dimensions to the instance size, so a policy trained on one size class cannot even be *evaluated* on another—at odds with the amortization argument that justifies learning in the first place.

One question we do not reopen. Intervals admit no natural total order, so any comparison of interval makespans embeds a modelling choice; we adopt the lexicographic ranking that a systematic study of the alternatives found most robust for this problem (Díaz, González-Rodríguez, Palacios, Díaz, & Vela, 2020), and keep it fixed throughout so that every method in this paper is judged by the same criterion.

This paper addresses both obstacles and asks three questions. *Q1:* can a constructive policy trained on a handful of interval instances compete with strong hand-crafted constructive baselines? *Q2:* can a single network, trained on one size class, remain useful across the whole benchmark without retraining? *Q3:* which standard ingredients of the JSP-DRL toolbox—attention encoders, multi-size training, automatic hyperparameter configuration—actually pay off at practical training budgets?

Contributions.

1. **A size-invariant, uncertainty-aware policy for the IJSP** (Section 4). All features are dimensionless: temporal quantities are scaled by a per-instance lower bound and interval uncertainty enters as relative widths. A Deep Sets (Zaheer et al., 2017) encoder over the eligible-operation set, combined with a pooled global context, yields policy *and* value networks of $\approx 1.2 \times 10^5$ parameters, independent of the numbers of jobs and machines.
2. **An empirical characterization** (Section 6) answering Q1–Q3: budget scaling on the training class, zero-shot dominance over the best dispatching rule on all seven Taillard size classes from a single network, and transfer to a second benchmark of classical instances.
3. **A comparison against both families of constructive heuristic** (Section 6.4), on the 70 benchmark instances and with per-instance results reported in full (Appendix A). The hand-crafted side is the traditional dispatching rules—SPT, LPT, MOR, MWKR and EST, plus two Giffler–Thompson variants—which a single greedy pass improves on instance by instance. The learned-symbolic side is a rule evolved by genetic programming for this same benchmark (Díaz Rodríguez, 2026b), a comparison Xu et al. (2025) single out as missing from the literature: the two paradigms are rarely measured against each other under a common protocol. Here the answer is budget-dependent rather than a verdict—pass for pass the evolved rule wins, and at 1024 samples each the policy does—which is itself the informative outcome.
4. **An analysis of what earns its cost** (Section 7): a permutation audit of which inputs the network consults, six controlled ablations (inference budget, attention versus pooling, specialist versus multi-size training, and three that delimit what interval awareness contributes) of which four are negative results, a Monte Carlo comparison of how faithfully the schedules execute, and a two-campaign automatic-configuration study whose racing optima repeatedly failed to transfer to the operating budget.

Section 2 reviews related work and Section 3 states the problem. Section 4 presents the policy, Section 5 the experimental methodology and Section 6 the results; Section 7 analyses them and states the limitations, and Section 8 concludes.

2 Related Work

2.1 Job shop scheduling under interval uncertainty

Uncertain processing times have been modelled with probability distributions, fuzzy numbers and intervals. The fuzzy JSP line is mature (González-Rodríguez et al., 2010; Palacios, González-Rodríguez, Vela, & Puente, 2015); the interval model can be read as its minimal-commitment limit, retaining only support bounds. Lei (2011) opened the interval line with population-based neighbourhood search; genetic algorithms with interval rankings (Díaz et al., 2020), robustness-oriented tardiness models (Díaz, Palacios, Díaz, Vela, & González-Rodríguez, 2022), and analytical treatments of special cases (Sotskov, Matsveichuk, & Hatsura, 2020) followed, and interval durations now

appear in richer shop variants as well (Zheng, Xiao, Zhang, & Lv, 2024). On the Taillard-derived benchmark used here, the strongest published results belong to elitist artificial bee colony algorithms (Díaz et al., 2023a, 2023b); these works also fixed the evaluation conventions this paper adopts (interval arithmetic, lexicographic ranking during construction, expected makespan for reporting).

2.2 Priority dispatching rules

The oldest constructive approach to the job shop builds a schedule in a single left-to-right pass, repeatedly choosing which of the eligible operations to place next. A *priority dispatching rule* makes that choice by scoring the candidates with a fixed formula over their attributes. The rules this paper uses as baselines and as verification anchors are the classical single-attribute ones, and we keep their usual acronyms. Two look at the operation itself: *shortest* and *longest processing time* (SPT, LPT) prefer the smallest and the largest duration. One looks at the schedule built so far: *earliest starting time* (EST) prefers the operation that could begin soonest given the current occupation of its machine and the progress of its job. Two look at the job the operation belongs to: *most work remaining* and *most operations remaining* (MWKR, MOR) prefer the job with the most pending work, counted in time units and in operations respectively. Under interval durations these attributes are intervals themselves, and candidates are compared with the interval ranking introduced in Section 3. Surveys catalogue well over a hundred variants of such rules (Haupt, 1989; Panwalkar & Iskander, 1977) and recent ones revisit those still in use (Đurašević & Jakobović, 2018).

A refinement worth separating from the rules themselves is the active-schedule generator of Giffler and Thompson (1960), which at each step restricts the candidates to the conflict set of the machine that would finish earliest, and applies a priority rule only to break ties within that set. The restriction guarantees *active* schedules—no operation can be started earlier without delaying another—and lifts the plain rules substantially; we write G&T-SPT and G&T-MWKR for the two combinations used here.

Their appeal is cost: one pass, no search, a solution in milliseconds. Their limit is that a fixed formula cannot suit every shop and objective (Haupt, 1989), which is the opening the learned rules of the next subsection address—and the reason a constructive policy is compared against these rules and not against per-instance metaheuristics, which belong to a different computational class.

2.3 Learning constructive heuristics for scheduling

Neural combinatorial optimization began with pointer networks (Vinyals, Fortunato, & Jaitly, 2015) and their RL training (Bello, Pham, Le, Norouzi, & Bengio, 2016); attention encoders became standard for routing (Kool, van Hoof, & Welling, 2019; Nazari, Oroojlooy, Snyder, & Takáč, 2018). For the deterministic JSP, Zhang et al. (2020) learn dispatching policies over disjunctive-graph GNN embeddings, Park et al. (2021) refine such representations and report cross-size generalization, and Tassel et al. (2021) contribute an efficient environment formulation. Sampling several rollouts and

keeping the best is a standard inference-time lever (Kool et al., 2019). Permutation-invariant set encoders were formalized as Deep Sets (Zaheer et al., 2017), of which attention is the natural strict generalization; our architecture starts from the simpler construction and evaluates attention as a controlled ablation.

Learning constructive heuristics is older than its neural incarnation: genetic programming (GP) hyper-heuristics evolve interpretable priority expressions over hand-crafted attributes (Branke, Nguyen, Pickardt, & Zhang, 2016; Nguyen, Mei, & Zhang, 2017). The survey by Xu et al. (2025) compares the GP and RL families for job shop scheduling and notes the scarcity of comparisons under a common protocol. A companion study (Díaz Rodríguez, 2026b) develops the GP side for the interval benchmark used here, which lets Section 6.4 put the two artefacts on the same instances at matched *inference* budgets. A controlled comparison of the two paradigms with *training* budgets matched as well is beyond the scope of either paper and is the subject of dedicated ongoing work.

3 Problem Statement

Definition 1 (IJSP) An IJSP instance comprises n jobs $J_1, \ldots, J_n$ and m machines. Job J_j is a fixed sequence of m operations $(o_{j1}, \ldots, o_{jm})$, where o_{jk} requires a specific machine ν_{jk} and an uncertain processing time known to lie in the interval $\mathbf{p}_{jk} = [p^L_{jk},\, p^U_{jk}]$, $0 \le p^L_{jk} \le p^U_{jk}$. Each machine processes at most one operation at a time and preemption is not allowed.

A solution is a processing order of all nm operations compatible with job precedences; we encode it as a *permutation with repetition* of job indices, where the k-th occurrence of j denotes o_{jk}. Given an order, its *semiactive schedule* starts every operation as early as its job and machine predecessors permit. With component-wise interval addition $[a,b]+[c,d] = [a+c,\, b+d]$ and join $\max([a,b],[c,d]) = [\max(a,c), \max(b,d)]$, starts and completions are intervals:

$$\mathbf{s}_o = \max\big(\mathbf{c}_{JP(o)},\, \mathbf{c}_{MP(o)}\big), \qquad \mathbf{c}_o = \mathbf{s}_o + \mathbf{p}_o, \tag{1}$$

where $JP(o)$ and $MP(o)$ are the job and machine predecessors of o. The interval makespan collects the job completions component-wise,

$$\mathbf{C}_{\max} = \Big[\max_j c^L_{o_{jm}},\; \max_j c^U_{o_{jm}}\Big], \tag{2}$$

and, because Eq. (1) is monotone in the durations, the executed makespan of *any* realization of the processing times falls inside $\mathbf{C}_{\max}$. Intervals carry no canonical total order, so schedule comparison requires a ranking choice; during construction and search we use the lexicographic order

$$\mathbf{a} \prec_{lex} \mathbf{b} \iff a^U < b^U \;\vee\; \big(a^U = b^U \wedge a^L < b^L\big), \tag{3}$$

which prioritizes the worst case, in line with min–max robust optimization and with the IJSP literature (Díaz et al., 2020, 2023a). For reporting we follow the field

convention (Díaz et al., 2020, 2023a):

$$\mathrm{RE}(\%) = \frac{E[\mathbf{C}_{\max}] - \mathrm{LB}}{\mathrm{LB}} \times 100, \qquad E[\mathbf{C}_{\max}] = \tfrac{1}{2}\big(C^{L}_{\max} + C^{U}_{\max}\big), \tag{4}$$

with LB the published lower bound of the crisp Taillard counterpart (Taillard, 1993)—an indicative, hardware-independent yardstick rather than a strict optimality gap, since the crisp JSP and the IJSP are different problems.

Remark 1 For instances whose intervals are symmetric perturbations of a crisp instance, the midpoint scenario coincides with the original crisp durations; comparisons under Eq. (4) are therefore scenario-consistent (midpoint solution quality against the midpoint-scenario bound).

4 Method

Reinforcement learning is learning to decide from experience. An *agent* interacts with an environment in steps: it observes the current *state* s, chooses an *action* a among those available, and receives a numeric *reward* signalling how good the consequences were. The agent's behaviour is its *policy* $\pi(a \mid s)$: a function that, given the state s, assigns a probability to each available action a. The policy is the object being learned—training nudges it so that actions followed by a high cumulative reward (the *return*) become more likely.

Policy-gradient methods such as PPO (Schulman, Wolski, Dhariwal, Radford, & Klimov, 2017) judge an action not by its return alone but by comparing that return with what was *expected* from the state where the action was taken; only the surplus—the *advantage*—moves the policy. The expectation is itself learned: a *value function* $V(s)$ estimates the return attainable from state s. In practice both functions share one network with two outputs, a *policy head* that scores the available actions and a *value head* that estimates $V(s)$, trained jointly. This is why the network in this paper has two heads.

Instantiated on scheduling, the agent is a dispatcher. The state is the partial schedule; the available actions are the *eligible* operations—the next unscheduled operation of each job, so choosing an action a means choosing which eligible operation i to schedule next, and we write the policy over candidates as $\pi(i \mid s)$; an *episode* is the construction of one complete schedule, and the reward rests on the makespan achieved. The trained policy then plays the same role as the priority dispatching rules of Section 2.2: where a hand-crafted rule orders the eligible operations by a fixed criterion, the policy orders them by preferences it has learned from experience. Everything around that choice (the schedule builder of Eq. (1), the interval arithmetic, the evaluation) is the same machinery every baseline in this paper uses.

Figure 1 shows the network, read left to right. Each eligible operation enters as a row of 16 numbers that describe it: how long it may take, how much work its job still carries, how soon it could start (Section 4.2). A small multilayer perceptron (MLP)—the *same* one for every candidate—turns each row into an *embedding*: a vector of h

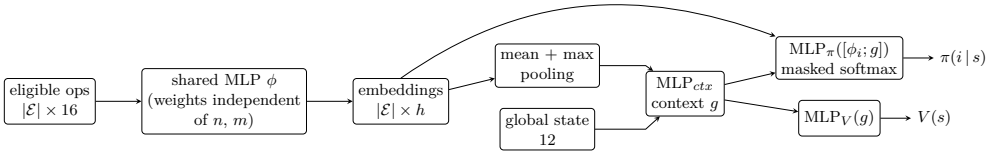

Fig. 1 The size-invariant network. Every eligible operation is embedded by the same MLP ϕ; mean and max pooling collapse the variable-size candidate set into fixed-size summaries that, joined with the global features, form the context g. The policy head scores each candidate embedding against g; the value head reads g alone. No dimension depends on the numbers of jobs or machines.

learned quantities that re-describes the candidate in terms the network finds useful. The number of candidates varies from step to step, so the embeddings are *pooled*: their mean and their componentwise maximum are two fixed-size summaries of the whole set (the typical candidate and the extreme one). Joined with 12 numbers describing the shop as a whole (the *global state*), these summaries are condensed by another MLP into a single *context* vector g—the network's picture of the current situation. The two heads of the previous paragraph read from here. The policy head looks at each candidate's embedding next to g and scores it; a *softmax* turns the scores into the probabilities $\pi(i \mid s)$ (*masked* means that unused padding slots are excluded). The value head reads g alone and outputs $V(s)$. In total, four small two-layer MLPs—the candidate encoder ϕ, the context combiner, and the two heads—composed into a single network and trained jointly end to end.

The rest of the section makes each stage precise: Section 4.1 states the construction loop formally (states, actions, rewards); Section 4.2 defines the 16+12 input quantities; and Section 4.3 the three network stages. How the network is trained, and how it is used once trained, belongs with the experimental setting and is described in Section 5.4.

4.1 Dispatching as a Markov decision process

Cast in the terms above, schedule construction is a finite-horizon Markov decision process of exactly nm steps. The *state* after t steps comprises the partial schedule (job pointers, job and machine completion intervals) and the set $\mathcal{E}_t$ of eligible operations—the next unscheduled operation of each unfinished job, so $1 \le |\mathcal{E}_t| \le n$. The *action* selects one element of $\mathcal{E}_t$; the transition applies Eq. (1). Episodes always terminate after nm steps.

The *reward* is a weighted sum of six shaping components, each collapsing intervals by their worst case (upper bound); component scales adapt automatically to each instance through its lower bound, and the weights were chosen on the development set of Section 5.1:

- *terminal makespan* (weight 1.0): the negative scaled makespan at episode end, plus a bonus when the schedule lands within 5% of the instance bound—the quantity actually being minimized;

Table 1 Per-operation features (all dimensionless). $d = \mathbf{p}_{jk}$, **est** = earliest start interval, **rem** = remaining work interval of the job after o; mid$(\cdot)$ is the interval midpoint; $L(\mu)$ the remaining worst-case load of machine μ; $\bar{L}$ its average over machines.

#	Feature	Definition
1–2	duration bounds	d^L/LB, d^U/LB
3	duration uncertainty	$(d^U - d^L)/\text{mid}(d)$
4–5	earliest start bounds	est^L/LB, est^U/LB
6	start uncertainty	$(est^U - est^L)/\text{mid}(\mathbf{est})$
7–8	remaining job work	rem^L/LB, rem^U/LB
9–10	job position	$(m-k-1)/m$, k/m
11	machine load	$L(\mu)$/LB
12	potential idle gap	$\max(0,\, c_J^U - c_\mu^U)$/LB
13	machine congestion	$L(\mu)/\bar{L}$
14	slack	$(est^U - \min_{o' \in \mathcal{E}} est_{o'}^U)$/LB
15–16	completions	c_μ^U/LB, c_J^U/LB

- *local improvement* (weight 0.3): the step change of the projected makespan, penalizing deteriorations twice as much as it rewards improvements—a dense signal so early decisions are not credited only nm steps later;
- *machine idleness* (0.15) and *load balance* (0.15): discourage gaps on machines and lopsided load, the classic proxies for a compact schedule;
- *operation criticality* (0.05) and *job progress* (0.05): small nudges towards operations on the emerging critical path and towards finishing what is started.

4.2 Dimensionless, uncertainty-aware features

Raw times do not travel between instances: a duration of 80 is long when the whole schedule spans 1,200 time units and short when it spans 3,000, so a network fed raw values would be tied to the scale it was trained on. The representation therefore expresses every temporal quantity as a fraction of the instance's lower bound LB (worst case if the bound is itself an interval), and every uncertainty as a ratio of interval width to magnitude. What the policy sees is thereby comparable across instances—and across sizes, which is half of the size invariance. Each eligible operation $o = o_{jk}$ on machine $\mu = \nu_{jk}$ is described by the 16 dimensionless quantities of Table 1; the global context is the 12-dimensional aggregate of Table 2. Two design rules follow from ablation evidence (Section 7.4): (i) features must add information that is *not* a linear combination of others (interval widths enter as relative, hence non-linear, magnitudes); and (ii) normalization must be co-designed with initialization and learning rates rather than retrofitted.

4.3 Size-invariant architecture

The remaining obstacle is that the number of candidates changes at every step and from instance to instance, while a neural network wants inputs of a fixed size. The

Table 2 Global context features (all dimensionless aggregates; no dimension depends on n or m).

Feature	Definition
progress	scheduled operations / nm
partial makespan	$\max_\mu c_\mu^U$/LB
machine completions	mean and std of c_μ^U/LB
remaining loads	mean, max and std of $L(\mu)$/LB
remaining job work	mean and max of job remaining /LB
eligibility	$\lvert\mathcal{E}\rvert/n$
pending uncertainty	$\sum$(widths)/ $\sum$(midpoints) of unscheduled ops
total load	$\sum_\mu L(\mu)/(\mathrm{LB}\cdot m)$

architecture dissolves this with two ideas: the *same* small network reads every candidate, so any number of them can be processed; and the resulting embeddings are *pooled* into fixed-size summaries, so the scoring stage can see the whole decision at once. Formally, let $x_i \in \mathbb{R}^{16}$ be the features of eligible operation i and $g_0 \in \mathbb{R}^{12}$ the global features; the construction has three stages (Figure 1).

Embed. The shared two-layer MLP ϕ (width h=128, LayerNorm, ReLU, orthogonal initialization) maps each candidate to an embedding $\phi_i = \phi(x_i)$. Because the same weights process every candidate, the layer accepts any number of them—this is where size invariance over jobs comes from.

Pool into a context. The variable-size set $\{\phi_1, \ldots, \phi_{|\mathcal{E}|}\}$ must be summarized into a fixed-size vector before anything global can be computed. Mean and max pooling provide two complementary permutation-invariant summaries (Zaheer et al., 2017)—the average candidate and the extreme one—which, concatenated with the global features g_0, are mixed into the context:

$$g = \mathrm{MLP}_{ctx}\big(\big[\tfrac{1}{|\mathcal{E}|}\textstyle\sum_i \phi_i;\ \max_i \phi_i;\ g_0\big]\big) \in \mathbb{R}^h. \tag{5}$$

This embed-then-pool pattern is the *Deep Sets* construction of Zaheer et al. (2017), whose central result is that *any* function invariant to the order of a set can be written in this form—so the simplicity costs no expressiveness in principle. The result tables refer to the base model by this name, in contrast to its attention variant (Section 4.4).

Score. The *policy head* rates each pair (candidate embedding, context)—"how good is operation i given the decision as a whole"—and a softmax over the true candidates (padding masked out) turns the scores into probabilities. The *value head* estimates the return attainable from the current state—the $V(s)$ of the advantage computation introduced above—and reads the context alone, since expectations concern the situation, not any one candidate:

$$\pi(i \mid s) = \mathrm{softmax}_i\, \mathrm{MLP}_\pi([\phi_i; g]), \qquad V(s) = \mathrm{MLP}_V(g). \tag{6}$$

No parameter dimension depends on n or m: the same $\approx 1.2 \times 10^5$-parameter network dispatches any instance. The last policy layer is initialized with gain 10^{-2}, so at the

start every candidate receives a nearly equal score and the untrained policy is close to uniform.

4.4 The attention variant

Pooling summarizes the candidate set by two statistics, its average and its extreme, and every candidate is then scored against that same summary. What this cannot express is a comparison between two *particular* candidates: whether operation i deserves the machine now may depend on which other eligible operation would claim it next, and a mean says nothing about which candidate that is. *Self-attention* is the standard remedy, and the ingredient the neural combinatorial optimization literature reaches for first (Section 2.3). In an attention layer every candidate emits a *query*, a *key* and a *value*; the similarity between one candidate's query and another's key decides how much of that other's value is added into the first's embedding. Each candidate is thereby rewritten as a content-weighted reading of all the others, with the weights computed from the embeddings themselves rather than fixed in advance.

We build this as a variant to be tested, not as part of the proposed model. A stack of B pre-layer-normalization transformer blocks transforms $\{\phi_i\}$ after the shared encoder and before pooling, that is, between the embedding and pooling stages of Figure 1. Each block normalizes the embeddings, applies multi-head attention over them (4 heads) and adds the result back, then applies a two-layer feed-forward network the same way; both additions are *residual*, that is, the block adds to its input instead of replacing it, so an untrained block behaves almost like the identity and the stack starts close to the base model. No positional encoding is used: the candidates form a set with no meaningful order, and numbering them would destroy the permutation invariance Section 4.3 was built to guarantee. Padding slots are masked out of the attention weights, so no candidate ever attends to an operation that is not there.

Attention is a flag on the same code path, and B=0 reproduces the base network exactly—which is what makes the comparison of Section 7.3 a controlled one, differing in this component and nothing else. Its price is not small. A single block carries $\approx 1.3 \times 10^5$ parameters—the query, key and value projections, the output projection, two layer normalizations and the feed-forward network—so the B=2 variant we evaluate grows the network from 1.2×10^5 to 3.9×10^5 parameters, a factor of 3.2, and makes an episode about 30% more expensive. Section 7.3 reports what that buys.

During batched training, variable-size candidate sets are padded to the batch maximum with a boolean mask applied to attention, pooling and logits; unit tests verify that masked-padded forward passes are numerically identical to unpadded ones.

5 Experimental Methodology

5.1 Instances, data split and metric

Experiments use the 70 interval Taillard instances TA1–TA70 that anchor the recent IJSP literature (Díaz et al., 2023a, 2023b): seven size classes from 15×15 to 50×20, each crisp duration replaced by an integer interval symmetric around it, with half-width drawn uniformly up to 15% of the original value (Díaz et al., 2020). The

Table 3 Machine and software used for every experiment in this paper.

Component	Specification
CPU	AMD Ryzen 5 3400G, 4 cores / 8 threads, 3.7 GHz
Memory	32 GB
GPU	NVIDIA GeForce RTX 5060 Ti, 16 GB (tuning only)
Operating system	Windows 10 Pro 22H2 (64-bit)
Language	Python 3.12.6
Learning	PyTorch 2.9.1, CUDA 13.0 build
Numerics	NumPy 2.3.5, SciPy 1.17.0
Figures	Matplotlib 3.10.8
Tuning	irace 4.4.3 on R 4.6.1 (Section 5.3)

benchmark is openly available (Díaz Rodríguez, 2026a). Of the 70 instances, four carry gradients: the final model trains on **TA11–TA14** (20×15) only. Six more of the same class, **TA15–TA20**, form the *development set*—never used for gradients, but every design and configuration decision was judged on their performance, so they are reported separately and excluded from claims about unseen data. The remaining 60 instances were touched by neither training nor development. Three independent training runs, differing only in their random seed, are reported as mean and best. Performance is measured by RE, Eq. (4), using the published crisp lower bounds.

5.2 Computational environment

Table 3 lists the machine and the software stack. Two remarks matter for reading the costs reported later. First, every number in this paper—training, evaluation, the dispatching-rule baselines and the ablations—was produced on the CPU. The network is small enough (Section 4.3) that a forward pass is negligible next to the environment step that follows it, so the GPU buys nothing for a single rollout; it was used only for the vectorized batched rollouts of the operating-fidelity tuning campaign (Section 5.3), where many episodes advance in lockstep and the batched forward pass does pay. The method therefore carries no dependence on specialized hardware, which is part of what makes the deployment argument of Section 6.4 credible.

Second, the wall-clock figures quoted there are measurements on this machine, whose four physical cores were at times shared by concurrent experimental arms. They are reported as orders of magnitude—hours to train, milliseconds to dispatch—and not as a timing benchmark; nothing in the paper's conclusions turns on them.

5.3 Hyperparameter configuration

All reported results use the hyperparameters of Table 4 *without problem-specific tuning*. To assess whether this leaves performance on the table, we ran an automatic configuration study with irace (López-Ibáñez, Dubois-Lacoste, Pérez Cáceres, Birattari, & Stützle, 2016). irace *races* candidate configurations: they are evaluated side by side on a growing set of instances (here, training seeds), statistical losers are dropped

Table 4 Hyperparameters, identical across every experiment reported in this paper. Most are standard PPO values; the discount factor and the batched update scheme depart from common practice for the reasons given in Section 5.4. None was tuned for this problem, though not for want of trying: two irace campaigns searched the space of Table 5 for better ones and neither transferred to the operating budget.

Hyperparameter	Value	Hyperparameter	Value
Optimizer		*PPO objective*	
learning rate (Adam)	$3 \cdot 10^{-4}$	clipping range	0.2
Adam ϵ	10^{-5}	GAE λ	0.95
gradient clipping	0.5	discount factor γ	1.0
Network and schedule		*PPO updates*	
hidden width h	128	epochs per update	4
weight initialization	orthogonal	minibatch size	256
policy update	every 4 ep.	entropy coefficient	0.01
greedy evaluation	every 10 ep.	advantage normalization	per update

Table 5 Search space of the two irace campaigns. Real parameters were sampled on a logarithmic scale, the rest from the listed values. Every default of Table 4 lies inside the space, so the race could have returned it. The second campaign fixed minibatch size and update period at their defaults, on which the elites of the first had agreed unanimously.

Parameter	Default	Campaign 1 (330 exp.)	Campaign 2 (300 exp.)
learning rate	$3 \cdot 10^{-4}$	$[10^{-4}, 10^{-3}]$	$[10^{-4}, 10^{-3}]$
entropy coefficient	0.01	[0.003, 0.03]	[0.003, 0.03]
clipping range	0.2	0.1, 0.2, 0.3	0.1, 0.2, 0.3
epochs per update	4	2, 4, 8	4, 8
GAE λ	0.95	0.90, 0.95, 0.98	0.90, 0.95, 0.98
hidden width	128	64, 128, 256	128, 256
minibatch size	256	128, 256, 512	fixed
update period	4 ep.	2, 4, 8 ep.	fixed

early, and the budget concentrates on the survivors, called the *elite* configurations. The race covered the 8-parameter space of Table 5 over a budget of 330 tuning experiments (309 raced) at reduced fidelity (1 training instance, 300 episodes, ∼8 minutes per evaluation), with training seeds as racing instances. The five elite configurations agreed on a coherent faster-learning regime (learning rate 7–9·10^{-4}, clip 0.3, minibatch 128, updates every 2 episodes, λ=0.90) and were insensitive to network width. However, at the full 1000-episode operating budget the best elite scored 14.5% mean RE versus 13.4% for the defaults (+1.66% mean makespan; five of six development instances within noise, one significantly worse) at ∼60% higher training cost: the low-fidelity optimum did not transfer.

To rule out that this non-transfer was an artifact of the reduced tuning fidelity, a second, *operating-fidelity* campaign raced 295 experiments of a 300 budget, each

Algorithm 1 Best-of-N inference

$(\sigma^*, \mathbf{C}^*_{\max}) \leftarrow$ greedy rollout of π
for $i = 1$ **to** $N - 1$ **do**
  $(\sigma, \mathbf{C}_{\max}) \leftarrow$ sampled rollout of π
  if $\mathbf{C}_{\max} \prec_{lex} \mathbf{C}^*_{\max}$ **then** $(\sigma^*, \mathbf{C}^*_{\max}) \leftarrow (\sigma, \mathbf{C}_{\max})$
  end if
end for
return σ^*

training at the full 1000 episodes on two instances (enabled by a ∼3× GPU-vectorized rollout implementation), over the reduced 6-parameter space of Table 5, seeded with the defaults and the low-fidelity elites. This campaign converged to a coherent region *distinct* from the defaults (clip 0.1, 8 epochs, λ=0.90, width 256, learning rate 5–9·10^{-4}) and eliminated the seeded default during racing. Yet when its winning configuration was retrained on the full four-instance set over three seeds and evaluated against the default checkpoints under an identical protocol, it again failed to improve: 14.73% versus 13.52% mean RE over the six development instances (+1.21 points; better on two instances, worse on four). Two conclusions follow: automatic configuration does not beat the defaults of Table 4 for this problem *even when performed at the operating budget*, so the reported results do not depend on fine tuning; and the recurring gap between racing-optimal and confirmation-optimal configurations is itself a cautionary observation for automatic DRL configuration, where single-run training cost is too noisy to separate near-equivalent configurations.

5.4 Training and inference

We train with PPO (Schulman et al., 2017); Table 4 lists all hyperparameters. Two choices depart from common JSP-DRL practice and are enabled by co-design with the normalized representation: a discount factor $\gamma = 1$ (undiscounted finite horizon—future rewards count in full; with the customary $\gamma = 0.99$ the terminal makespan signal reaches early decisions attenuated by $0.99^{nm} \approx 0.05$ for 20×15 instances) and batched minibatch updates with advantage normalization. Training on a set of instances proceeds in blocks (one instance at a time) with full weight transfer. A *rollout* is one complete construction episode played by the current policy: *sampled* if each operation is drawn from π, as during training, or *greedy* if every step takes the most probable one. Sampled training episodes are interleaved with periodic greedy evaluations, and the best solution and best saved weights (the *checkpoint*) are tracked across both.

Best-of-N inference.

At evaluation time we exploit the stochastic policy: one greedy rollout plus $N-1$ sampled ones, keeping the best solution under the ranking of Eq. (3) (N=64). This choice is motivated empirically in Section 7.2: during training, the best solutions frequently originate from sampled episodes, i.e. the policy is more valuable as a *distribution* than as its argmax—consistent with sampling-based decoding in neural combinatorial optimization (Kool et al., 2019).

Table 6 RE (%) per development instance, mean [best] over 3 seeds, Deep Sets model, best-of-64 inference.

Budget	TA15	TA16	TA17	TA18	TA19	TA20
100 eps	34.1 [31.9]	26.8 [22.8]	28.2 [24.9]	26.2 [24.9]	29.2 [24.1]	24.3 [22.4]
300 eps	18.7 [17.1]	16.1 [15.4]	14.6 [13.3]	16.9 [16.4]	16.3 [15.5]	16.8 [16.1]
1000 eps	**13.7** [12.1]	**12.6** [11.8]	**13.0** [11.8]	**15.9** [14.6]	**12.1** [11.6]	**13.4** [12.2]
MOR	51.3	35.8	50.1	54.2	43.2	43.7

5.5 Baselines

Constructive baselines are the dispatching rules of Section 2.2—SPT, LPT, MOR and MWKR, of which MOR is uniformly the strongest, plus EST—and the two Giffler–Thompson combinations. G&T-MWKR is the strongest deterministic constructive baseline on this benchmark: 29.5% mean RE across the 70 instances, versus 42.3% for EST—the strongest plain rule overall, though weaker than MOR on the 20×15 training class—and 45.5% for MOR (the SPT tie-break variant performs poorly, 70.6%). Published IJSP metaheuristics (fEABC (Díaz et al., 2023b) and ESABC (Díaz et al., 2023a); 30 runs and minutes of per-instance search on dedicated hardware) are included as reference yardsticks from a different computational class rather than as direct competitors.

6 Results

6.1 In-size performance and budget scaling

Table 6 reports per-instance RE on the development set for increasing training budgets, and Figure 2 the aggregate picture. Three observations: (i) the policy scales monotonically with budget (28.1% → 16.6% → 13.4% mean RE at 100/300/1000 episodes per instance) with clearly diminishing returns at the last step, locating the pure-budget ceiling near 11–12%; (ii) seed variance collapses with budget (per-instance makespan std of 6–22 units at 1000 episodes), indicating consistent learning rather than lucky sampling; (iii) the gap to the best dispatching rule is a factor ≈3.5.

6.2 Zero-shot cross-size generalization

The size-invariant design allows direct evaluation of the 20×15-trained network on any class. Evaluation is *zero-shot*: the trained weights are applied to sizes they never saw, with no additional training or fine-tuning of any kind. Table 7 reports zero-shot RE on unseen instances of all seven classes: every seed's network outperforms MOR on *every* tested instance, with graceful degradation (means from ≈10% on 15×15 to 25.9% on the hardest transfer, 30×20) and no failure mode up to 1000-operation episodes. Notably, transfer quality improves with in-size training budget (mean over

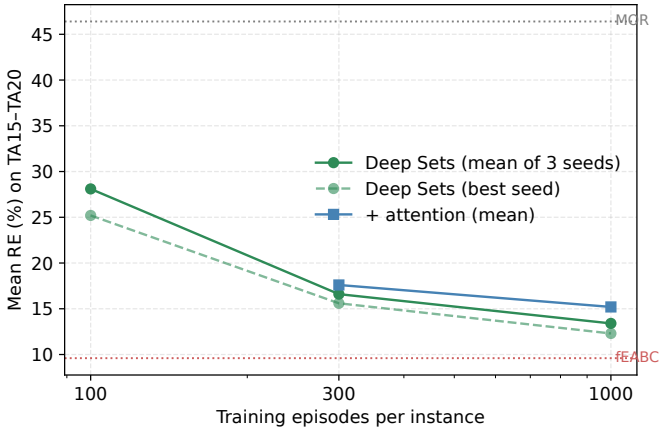

Fig. 2 Mean RE on the development set versus training episodes per instance. Dotted lines: best dispatching rule and published metaheuristics (different computational class) as yardsticks.

Table 7 Zero-shot RE (%) of the 20×15-trained network (best-of-64), mean [best] over the three training seeds, versus MOR.

Instance	Policy	MOR	Instance	Policy	MOR
TA1 (15×15)	13.1 [11.0]	29.7	TA31 (30×15)	15.8 [13.9]	34.4
TA2 (15×15)	9.7 [7.0]	34.8	TA41 (30×20)	25.9 [24.5]	53.5
TA5 (15×15)	10.5 [8.8]	50.7	TA51 (50×15)	15.3 [12.9]	38.2
TA21 (20×20)	13.2 [11.1]	40.8	TA61 (50×20)	16.8 [15.5]	48.7
TA25 (20×20)	15.0 [14.4]	53.2			

seeds: 30×20 from 30.3% with the 300-episode checkpoints to 25.9% with the 1000-episode ones; 50×15 from 22.2% to 15.3%), i.e. the *specialist*—our shorthand for the network trained on the single 20×15 class—learns structure that is not size-bound.

6.3 Cross-family generalization: the classical instances

The IJSP literature also reports on interval versions of twelve classical instances (FT10 and FT20, seven of the La family, and ABZ7–9), generated from their crisp counterparts with the same symmetric scheme as the Taillard set (Díaz et al., 2020). None of these families was seen during training or development, so they double as a cross-family generalization test. Following the literature, RE is measured here against the best-known expected-makespan values (Díaz et al., 2023b)—a tighter reference than Taillard's crisp bounds, so Table 8 is not comparable with the Taillard tables.

Table 8 positions the policy (best-of-64, mean and best of the three seeds) against the shared-evaluator rules and the published 30-run averages of the per-instance metaheuristics. The policy improves on EST and G&T-MWKR on all twelve instances and lands at 12.4% mean RE—between the hand-crafted constructive baselines (30.1% for

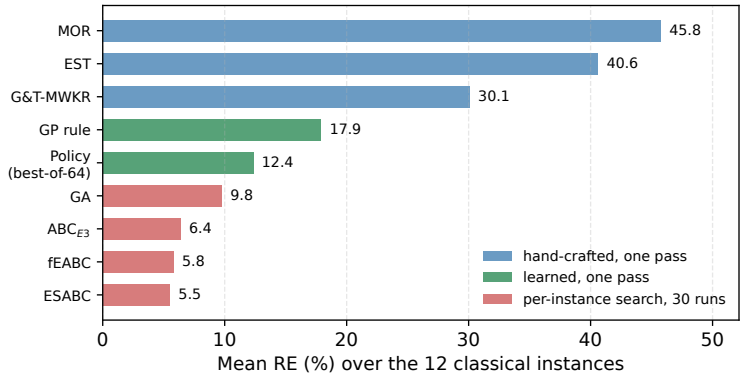

Fig. 3 Mean RE over the 12 classical interval instances, by computational class. The policy occupies the gap between what a single constructive pass had achieved and what per-instance search achieves, at the cost of the former.

Table 8 RE (%) on the 12 classical interval instances (cross-family zero-shot). Policy: best-of-64, mean over the three seeds; Policy^b: best seed. GP is the featured evolved rule of the companion study (Díaz Rodríguez, 2026b), one pass like the other constructive methods. Published metaheuristics are 30-run averages. Reference bounds are the best-known expected-makespan values, a different scale from Taillard's crisp bounds.

Inst.	EST	G&T	GP	Policy	Policy^b	GA	ABC$_{E3}$	fEABC	ESABC
FT10	32.6	32.2	20.4	8.5	6.3	5.2	4.1	3.5	3.0
FT20	27.7	40.0	13.2	5.8	4.4	4.4	1.7	1.8	1.8
La21	40.2	24.1	15.0	12.6	10.4	5.0	5.0	4.2	4.0
La24	42.2	26.3	10.1	11.9	10.9	6.3	5.1	4.9	5.0
La25	36.6	16.9	11.8	9.6	9.2	5.1	3.9	3.4	2.7
La27	47.2	34.8	25.0	11.3	10.9	10.2	4.7	4.6	4.1
La29	47.9	24.5	13.4	16.6	14.0	14.2	8.6	7.4	7.0
La38	54.8	27.0	15.0	12.8	11.7	9.2	6.9	6.1	5.8
La40	20.9	27.9	13.3	9.3	8.1	8.7	4.2	4.6	4.1
ABZ7	51.6	28.7	19.7	13.4	12.0	12.5	7.3	6.6	6.7
ABZ8	43.0	36.9	24.7	17.3	16.9	18.5	12.1	11.1	10.9
ABZ9	42.7	41.8	33.4	19.2	18.8	18.0	13.1	11.6	11.2
Mean	40.6	30.1	17.9	12.4	11.1	9.8	6.4	5.8	5.5

G&T-MWKR) and the per-instance searchers (GA at 9.8%, fEABC at 5.8%, ESABC at 5.5%), and ahead of the evolved rule of the companion study (17.9%), which occupies the same one-pass computational class. On La40, ABZ7 and ABZ8 its best seed edges past the GA average itself, with single-pass inference and no retraining. Figure 3 orders the methods by mean RE and colours them by computational class: the policy sits alone between the two.

6.4 Position among constructive methods

Training the final model takes 66–123 minutes per seed across the three seeds (4×1000 episodes) on the commodity CPU of Table 3. The cost is dominated by environment interaction, not by the small network: a seed dispatches 1.2 million operations, each paying for feature extraction over the unscheduled work, and the GPU-vectorization of rollouts in Section 5.3 bought its ∼3× precisely there. Constructing a solution takes milliseconds (greedy) to a few seconds (best-of-64 on 50×20). Against the hand-crafted rules the comparison is one-sided: a single greedy pass improves on MOR, EST and G&T-MWKR on each of the 70 instances (paired Wilcoxon signed-rank test, $p < 10^{-12}$; the matched-pairs rank-biserial correlation, an effect size that reaches 1 only when one side wins every single pair, is exactly 1). Raising the inference budget to best-of-1024 brings the full-benchmark mean RE to 13.0% over the 70 instances (Table 9 and Figure 5)—ahead of every deterministic constructive baseline on every instance, again with rank-biserial correlation 1, and within 2.5–5.1 points of the fEABC 30-run class averages.

Against the evolved rule of the companion study the comparison is not one-sided, and its answer depends on the inference budget the two are given (Figure 4). Pass for pass, the rule wins: one greedy pass of the policy improves on it in only 21 of the 70 instances, a median deficit of 2.0 points ($p = 4.6 \times 10^{-4}$), and averages 19.4% against the rule's 17.7%. Given 1024 samples each—the companion randomizes its rule with ϵ-greedy dispatching for exactly this purpose—the ordering reverses, though by less than the two headline averages alone would suggest: the policy wins 46 of 70 with a median advantage of 1.3 points ($p = 1.4 \times 10^{-3}$), 13.0% against 14.1%.

The two halves say the same thing from opposite sides. A single decision sequence drawn greedily from 1.2×10^5 parameters is worse than a compact evolved formula, so the network's advantage does not lie in its argmax; what it provides, and the formula does not, is a distribution whose samples improve faster with budget—the reading the best-of-N ablation of Section 7.2 reaches from within. Both learners saw the same four training instances, but under different regimes and costs, so this compares published artefacts rather than a controlled study; the latter, with training as well as inference budgets matched, is the subject of dedicated ongoing work. The policy's standalone RE (13.4% in-size; 10–26% zero-shot) thus sits between dispatching rules (42.3% for the best of them, EST) and per-instance metaheuristics (9.4% mean RE for fEABC over 30 runs)—establishing, for the IJSP, the computational-class trade-off documented for the deterministic JSP (Zhang et al., 2020).

7 Analysis

The results above establish what the policy achieves. This section asks what is doing the work: which inputs the network actually consults, which of the design choices behind it earn their cost, and how dependable the schedules prove when the durations finally materialize.

Fig. 4 The DRL policy against the evolved rule of the companion study (Díaz Rodríguez, 2026b) at matched inference budgets, one point per instance over the 70 interval Taillard instances. Points below the diagonal are instances where the policy wins. Left: one constructive pass each. Right: 1024 samples each, the rule randomized with ϵ-greedy dispatching.

Table 9 Mean RE (%) per size class over the 70 interval Taillard instances. Constructive methods are a single deterministic pass unless stated; the GP rule is the featured evolved rule of the companion study (Díaz Rodríguez, 2026b), also one pass; fEABC is a 30-run average on dedicated hardware, quoted as a reference from a different computational class. The 20×15 column (†) is the training and development class: it contains the four instances the policy trained on and the six on which the design decisions were made, and is therefore not a generalization result; the other six classes are untouched.

Method	15×15	20×15†	20×20	30×15	30×20	50×15	50×20	All
EST	32.3	49.2	41.5	42.4	56.6	33.2	41.0	42.3
MOR	41.4	46.6	47.2	44.1	58.8	34.1	45.9	45.5
G&T-MWKR	26.8	28.9	30.9	33.6	36.8	23.9	25.5	29.5
GP rule	15.7	16.6	18.1	21.1	24.1	13.8	14.6	17.7
Policy (greedy)	16.9	18.3	18.5	21.2	26.1	15.8	18.8	19.4
Policy (best-of-1024)	**8.8**	**11.2**	**12.9**	**15.0**	**19.6**	**10.4**	**13.4**	**13.0**
fEABC (30 runs)	5.9	8.7	10.3	9.8	16.4	5.7	9.0	9.4

7.1 What the policy attends to

The policy is not interpretable the way a priority expression is, but its inputs can be audited. Figure 6 reports a permutation test: at every decision of a greedy rollout, one feature column is shuffled across the eligible operations, and the resulting mean

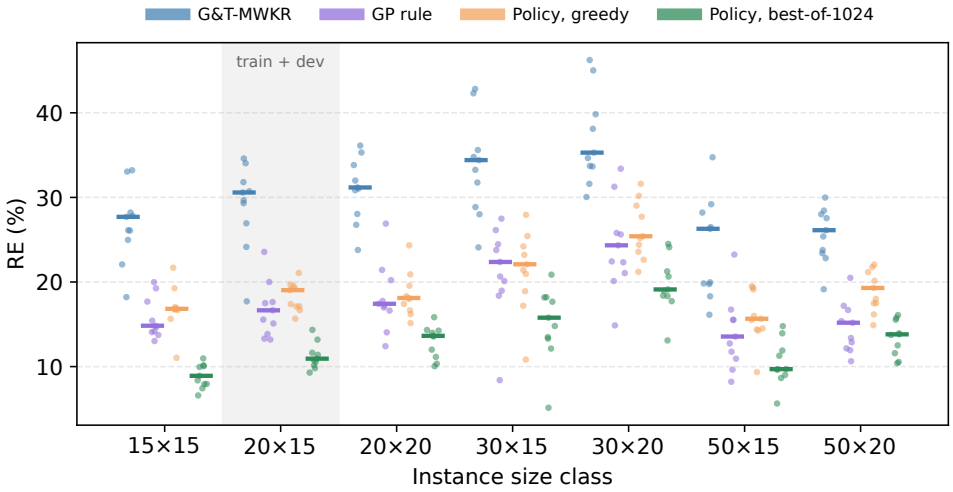

Fig. 5 Per-instance RE by size class over the 70 interval Taillard instances: every instance is a point, bars mark class medians. The 20×15 class is shaded because it holds the four training and six development instances; the other six classes were touched by neither.

RE over the development set and the three seeds is compared with the intact rollouts (18.2%).

Three features carry the policy: the relative slack (whose permutation adds 51.0 points of RE), the number of remaining operations in the job (+28.1), and the job's worst-case remaining work (+10.6)—the attribute families behind MOR and MWKR, and the very attributes the companion study's evolved expressions select (Díaz Rodríguez, 2026b). The interval-width features contribute nothing measurable at inference (−0.1 to +0.2 points): whatever the policy does with uncertainty, its decisions rest on worst-case magnitudes rather than on interval widths per se.

7.2 Ablation: best-of-N inference

Introducing best-of-64 inference (over one greedy rollout) improved the full-benchmark mean makespan by 3.6%, strictly better on 3 of 6 development instances and worse on none, while sharply reducing seed variance. Diagnostically, before this change the best training solutions frequently arose in early, exploratory episodes and were never surpassed by the greedy policy—evidence that the policy's value concentrates in its sampling distribution.

7.3 Ablation: attention versus pooling

The two-block attention variant of Section 4.4 was evaluated at both operating points with paired seeds: +0.9% mean makespan at 300 episodes (all six instances within the noise threshold) and +1.5% at 1000 (one instance significantly worse), for 3.2 times the parameters and ≈30% higher per-episode cost (Table 10). At these budgets the

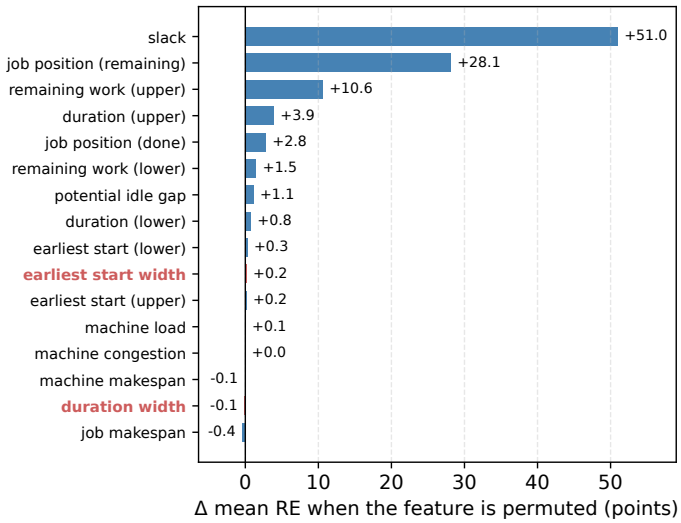

Fig. 6 Permutation importance of the 16 operation features: change in mean development-set RE when the feature's column is shuffled across the eligible operations at every decision (greedy rollouts, three seeds; intact baseline 18.2%). The two interval-width features are highlighted.

Table 10 Attention ablation: mean [best] RE (%) on the development set.

Configuration	300 eps	1000 eps
Deep Sets	**16.6** [15.6]	**13.4** [12.3]
+ attention (B=2)	17.6 [16.3]	15.2 [14.0]

pooled context already conveys the interaction information the policy can exploit; richer pairwise structure may require larger budgets or instance distributions to pay off.

7.4 Ablation: specialist versus multi-size training

Size invariance enables training on mixed size classes, a natural hypothesis for improving transfer. We trained agents on twelve instances spanning 15×15, 20×15 and 20×20, at matched total budget (3000 episodes) and at equalized *per-instance* budget (12,000 episodes—three times the specialist's total cost), against the 20×15 specialist, all sides with best-of-64 inference and averaged over their three seeds. At matched total budget the specialist wins on all five common evaluation instances (10.5 vs. 15.9 on 15×15; 15.0 vs. 20.0 on 20×20; 25.9 vs. 30.6 on the unseen 30×20). Tripling the multi-size budget recovers two of the five (9.6 vs. 10.5 on 15×15; 15.8 vs. 16.8 on the unseen 50×20) but not the rest: the specialist stays ahead on 20×20 (15.0 vs. 19.4), a class the multi-size agent itself trained on, and on both 30×15 and 30×20.

At matched cost, per-instance training depth dominates size diversity; even at triple cost, diversity buys back a minority of instances. Earlier development iterations provide the complementary caution: appending features that are linear combinations of existing ones, or retrofitting input normalization onto a system co-adapted to raw scales, degraded performance by 9–113%; representation changes must be co-designed with optimization.

7.5 What interval awareness contributes

The representation of Section 4.2 was designed to make uncertainty visible: interval widths enter as relative magnitudes, and the durations, earliest starts and remaining work of a candidate appear as bounds rather than as point estimates. The permutation audit of Section 7.1 already found the two width features unused at inference. Three training-time ablations ask the stronger questions: whether the policy would be any worse had it never been able to see the uncertainty at all, and whether it would use the widths if the reward paid for them.

The first removes the information. The encoder collapses every interval to its worst case before the features are computed, so the width features are identically zero and the bounds coincide; the architecture, the hyperparameters and the schedule builder are untouched, and the environment still executes interval semantics. Three seeds retrained from scratch reach 13.62% mean RE on the development set against the 13.44% of the full representation—a difference of 0.18 points, in the paired comparison over the eighteen instance-seed pairs worse on twelve and better on six, and not significant (Wilcoxon signed-rank, $p = 0.47$).

The second removes the uncertainty from training altogether. Because the benchmark perturbs each crisp duration symmetrically, the original crisp instances *are* the midpoint scenario of their interval counterparts, which makes them available as a control: we train on them and evaluate, unchanged, on the interval instances. The resulting policies reach 14.18%, again indistinguishable from the interval-trained ones ($p = 0.28$). Training under interval arithmetic buys nothing that training on the midpoints does not already provide.

Taken together with the permutation audit, the delimitation is consistent at three levels: the widths are unused at inference, removing them costs nothing, and the uncertainty need not be present during training either. For expected makespan the interval machinery of this paper is inert. That is a bounded claim, and its boundary is the objective: the reward of Section 4.1 collapses every interval to its upper bound, so the policy is never asked to prefer a *narrow* makespan interval over a low one. Under such an objective—which has no counterpart in the crisp problem—the companion study finds the width terminals do earn their place (Díaz Rodríguez, 2026b).

The third ablation therefore moves the boundary itself. We retrain the policy, unchanged in every other respect, under

$$f_\lambda \;=\; C^U_{\max} + \lambda \left(C^U_{\max} - C^L_{\max}\right), \qquad \lambda = 1, \tag{7}$$

which charges the schedule for the width of the interval it predicts and is the analogue of the robust objective of the companion study. Because optimizing one quantity

and selecting by another would be incoherent, the best-of-N inference of this arm ranks its samples by f_λ as well. Deployed as that package, the objective does what it promises: over the same eighteen instance–seed pairs the interval narrows from 12.85% to 11.90% of the expected makespan (narrower on 13 of 18, $p = 0.010$) and the expected makespan itself rises by 1.94 points, to 15.38% ($p = 0.007$). Unlike the two ablations above, both effects are significant. A width-rewarding objective is available, and it costs about two points of RE.

The interesting part is where the narrowing comes from. Training and selection changed together above, so we separate them: from a fresh pool of 64 rollouts per instance and seed we reconstruct the best-of-64 of *both* arms under one common criterion, so that only the weights differ. Under that control the two policies become indistinguishable—ranking by the upper bound, the widths are 12.80% for the default arm and 12.69% for the robust one ($p = 0.93$); ranking by f_λ, 11.74% and 12.20% ($p = 0.12$), if anything the wrong way round. What narrows the interval is which sample is kept, not which weights produced it: applying the robust criterion to the *default* policy's own pool already buys most of the reduction, at a comparable cost in RE. The network, in other words, is not merely indifferent to the widths in its input; it does not become sensitive to them when the reward asks it to. That completes the delimitation at a fourth level, and it leaves Section 8 a sharper question than the one it would otherwise inherit: whether any training signal reaches these features, or whether the constructive setting itself decides the width before the policy has a say.

7.6 Executional robustness

A schedule is eventually executed under one realization of the durations, and its interval prediction is useful insofar as the executed makespan stays close to the reported expected value. We adopt the field's $\bar{\varepsilon}$ measure (Díaz et al., 2020; Leon, Wu, & Storer, 1994): for a fixed processing order with predicted makespan $\mathbf{C}_{\max}$,

$$\bar{\varepsilon} = \frac{1}{K} \sum_{k=1}^{K} \frac{\left| C_{\max}^{\mathrm{ex},k} - E[\mathbf{C}_{\max}] \right|}{E[\mathbf{C}_{\max}]}, \tag{8}$$

where $C_{\max}^{\mathrm{ex},k}$ is the crisp makespan of executing the order under the k-th realization, durations drawn independently and uniformly within their intervals. We use K=1000 common random numbers per instance—identical realizations for every method—on the fifteen instances of Sections 6.1–6.2, and report $\bar{\varepsilon} \times 10^3$. Every execution falls inside the predicted interval, as the monotonicity of Eq. (1) guarantees.

The constructive methods separate into two groups, and the policy is in the better one without leading it (Figure 7). Its schedules deviate by $\bar{\varepsilon} \times 10^3 = 6.2$ under greedy inference and 6.2 under best-of-64—sampling for a lower worst case neither helps nor harms predictability—against 6.9 for G&T-MWKR and 7.1 for MOR, and those two gaps are systematic (the policy is more faithful on 15/15 and 13/15 instances respectively, paired Wilcoxon signed-rank test, $p < 0.001$ and $p = 0.003$). Against the other two the policy is indistinguishable: EST deviates by 6.3 (8/15, $p = 0.60$) and

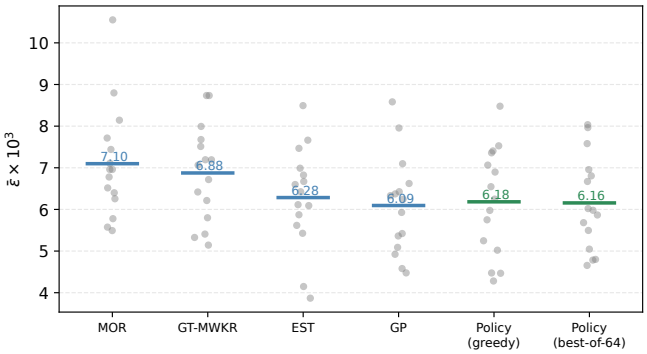

Fig. 7 Executional robustness by method: mean $\bar{\varepsilon} \times 10^3$ (bars) over the fifteen evaluation instances, each shown as a point. Lower is a more faithful a-priori prediction. EST, the evolved rule and the policy are indistinguishable at the top; MOR and G&T-MWKR are not.

the evolved rule of the companion study by 6.1 (7/15, $p = 0.98$), a hair ahead of the policy and well inside the noise.

Three methods therefore share the top of this ranking—a hand-crafted rule, an evolved formula and a neural policy—and they have one thing in common: none of them optimizes the width of the interval it predicts. A schedule can deviate little without being short, so faithfulness has to be pursued deliberately rather than inherited from a makespan objective; the companion study reaches the same conclusion from its side, where its rule likewise improves on G&T-MWKR but not on EST until an objective that penalizes width is introduced (Díaz Rodríguez, 2026b). Section 7.5 returns to this, and Section 8 states what it implies for the reward used here.

7.7 Limitations

Four limitations bound the scope of these conclusions. First, the benchmark follows the symmetric ±15% generation scheme standard in the IJSP literature; asymmetric intervals, and distributions in which the uncertainty varies sharply from one operation to another, would separate the worst-case and expected-value criteria far more than they do here, and no policy was trained under them. Second, the policy builds semiactive schedules, so the space it searches excludes the active schedules that the conflict-set restriction reaches and the insertion-based ones beyond them; the strongest constructive baseline of this paper exploits the first of those and the policy still leads it, but the ceiling is real. Third, three training seeds support means, bests and a variance estimate, which is what we report; they do not support significance testing across seeds, so the seed-level comparisons of Section 7 are stated as effects and not as tests. Finally, the comparison against the evolved rule of Section 6.4 matches inference budgets but not training ones, and it compares two published artefacts rather than two paradigms trained under one protocol; that study, which is what Xu et al. (2025) call for, and the use of the policy's sampling distribution to seed population-based search, are the subject of dedicated ongoing work.

8 Conclusions and Future Work

The three questions posed in the introduction now have answers. A constructive policy trained by PPO on four interval instances beats the best hand-crafted constructive baselines by a wide margin on its training class (Q1: 13.4% versus 29.5% for G&T-MWKR and ≈46% for the best plain rule). Because the representation is dimensionless and the encoder permutation-invariant, one network serves the entire benchmark: evaluated zero-shot, it outperforms the best dispatching rule on every tested instance of all seven size classes, and the same weights improve on every rule on all twelve classical instances of the FT, La and ABZ families (Q2). Against a rule evolved by genetic programming the answer is not one-sided but budget-dependent: pass for pass the rule is better, and the policy overtakes it only when both are given 1024 samples (Section 6.4), so what the network contributes over a compact formula is its sampling distribution rather than its argmax. And of the standard toolbox, only best-of-N inference paid off at practical budgets: self-attention did not improve on pooling, multi-size training did not overtake the single-size specialist even at three times its cost, and two independent automatic-configuration campaigns failed to beat the untuned defaults at the operating budget (Q3)—negative results we document with the same care as the positive ones. Three more delimit the interval representation that motivated the design: for expected makespan it turns out to be inert, since the policy neither uses the width features at inference (-0.1 points when they are shuffled), nor suffers when they are removed from training (13.62 against 13.44%, $p = 0.47$), nor needs the uncertainty present during training at all (14.18% when trained on the crisp midpoint instances, $p = 0.28$). Retraining under an objective that charges for the width closes the case rather than opening it: the predicted interval does narrow (11.90 against 12.85%, $p = 0.010$) at a cost of 1.94 points of RE, but with the selection criterion held fixed the robust weights and the default ones are indistinguishable—what narrows the interval is which sample is kept, not which network produced it. At execution time the policy's schedules are the most dependable of the constructive pool, matching EST, the robustness benchmark that evolved rules also fail to beat; and a permutation audit shows the network leaning on slack, remaining operations and worst-case remaining work—the attributes hand-crafted and evolved rules exploit—while ignoring the width features at inference.

Three directions follow. The first is suggested by the ablation of Section 7.4: since a specialist transfers so well and multi-size training buys so little, the productive use of size invariance is probably not to train on many sizes at once but to fine-tune a specialist onto a new class cheaply, a curriculum question this paper leaves open. The second concerns the representation, and the robust arm of Section 7.5 sharpens rather than settles it: paying for the width in the reward narrows the schedules the policy *selects* without measurably changing the policy itself, so the width features remain unconsulted even when the objective would reward consulting them. Whether the gradient simply never reaches them, or whether a semiactive constructive decision determines the width before the policy has any say in it, separates two very different diagnoses and calls for a different experiment than any run here—a reward defined on the width alone would tell them apart. The third is deployment: a policy that prices

an instance in milliseconds is a natural generator of initial solutions for population-based search, and the quality of its sampling distribution, documented here at a fixed budget, is what such a use would exploit.

Appendix A Per-instance results

Table A1 lists, for each of the 70 interval Taillard instances, the crisp lower bound and the RE (%) of the earliest-start rule, of Giffler–Thompson with MWKR tie-breaking, of the evolved rule of the companion study (Díaz Rodríguez, 2026b), and of the policy at its two inference budgets: greedy, averaged over the three training seeds, and best-of-1024. The ten 20×15 instances TA11–TA20 are the training and development set.

Table A1 Per-instance RE (%) of the constructive methods and of the policy. LB is the crisp Taillard lower bound; Pol. is the greedy policy and Pol.* the best-of-1024 policy.

Inst.	LB	EST	G&T	GP	Pol.	Pol.*	Inst.	LB	EST	G&T	GP	Pol.	Pol.*
TA1	1231	49.9	26.1	19.3	16.8	11.0	TA36	1819	41.1	42.8	23.8	18.9	13.3
TA2	1244	24.9	27.8	14.2	19.3	6.6	TA37	1771	43.4	35.6	18.4	24.2	17.7
TA3	1218	33.0	33.2	14.1	16.8	10.1	TA38	1673	45.0	28.0	20.1	23.2	14.8
TA4	1175	32.0	27.7	13.0	21.7	10.0	TA39	1795	46.6	28.9	22.4	17.2	13.5
TA5	1224	27.2	33.0	14.7	17.0	8.4	TA40	1651	46.9	42.3	26.1	27.9	20.9
TA6	1238	26.8	18.2	13.7	11.0	7.4	TA41	1906	44.6	45.0	33.4	31.6	24.5
TA7	1227	30.7	26.1	17.7	16.7	7.9	TA42	1884	71.1	38.1	24.3	25.4	18.3
TA8	1217	23.7	25.0	14.8	16.8	7.9	TA43	1809	68.7	39.8	22.3	29.0	20.6
TA9	1274	37.7	28.2	20.0	15.6	8.9	TA44	1948	52.1	34.7	25.8	22.6	19.0
TA10	1241	37.0	22.1	15.4	16.9	10.2	TA45	1997	50.2	33.7	14.9	21.2	13.1
TA11	1357	56.2	30.7	17.5	19.5	10.6	TA46	1957	60.8	31.6	22.4	23.6	17.7
TA12	1367	48.8	34.6	17.6	17.1	10.9	TA47	1807	46.8	30.1	25.6	25.2	19.1
TA13	1342	60.2	26.9	13.3	19.3	13.2	TA48	1912	59.1	33.7	21.1	24.4	18.4
TA14	1345	39.5	29.7	13.2	17.4	9.3	TA49	1931	51.0	35.3	20.1	27.7	21.3
TA15	1339	48.1	30.6	16.7	19.7	11.4	TA50	1833	61.4	46.2	31.3	30.2	24.1
TA16	1360	45.2	34.0	15.6	17.1	10.8	TA51	2760	33.8	34.7	23.2	19.1	11.9
TA17	1462	58.9	17.7	13.9	15.7	9.8	TA52	2756	38.0	26.5	12.7	16.0	11.3
TA18	1377	43.8	31.8	23.6	21.1	14.3	TA53	2717	34.1	18.3	11.8	15.7	9.7
TA19	1332	45.8	29.3	20.0	16.7	11.6	TA54	2839	20.9	16.1	8.2	9.3	5.6
TA20	1348	45.7	24.1	15.1	19.0	10.2	TA55	2679	31.7	29.2	15.5	19.4	14.8
TA21	1642	33.4	28.0	20.2	16.6	11.1	TA56	2781	34.8	19.8	10.9	14.4	8.6
TA22	1561	44.2	30.8	17.4	20.9	14.3	TA57	2943	32.9	20.0	9.6	15.5	9.6
TA23	1518	42.8	35.3	17.8	16.2	13.6	TA58	2885	37.4	26.3	13.6	14.5	9.0
TA24	1644	56.3	23.8	14.1	15.1	10.3	TA59	2655	26.6	28.2	16.7	19.5	13.9
TA25	1558	45.8	33.8	16.6	19.6	14.0	TA60	2723	41.7	19.8	15.5	14.3	9.7
TA26	1591	42.8	31.2	26.9	18.0	14.3	TA61	2868	45.7	25.4	15.4	20.1	13.9
TA27	1652	50.0	31.1	21.4	17.4	13.6	TA62	2869	43.0	27.6	17.2	21.2	15.5
TA28	1603	36.8	26.8	12.4	18.1	10.0	TA63	2755	38.8	23.8	13.4	17.5	13.7
TA29	1583	28.8	32.0	17.2	18.3	12.0	TA64	2702	38.2	28.4	12.2	17.5	11.6
TA30	1528	34.2	36.1	17.0	24.3	15.8	TA65	2725	42.9	30.0	20.5	22.1	16.1
TA31	1764	30.7	34.4	24.5	21.4	12.1	TA66	2845	41.8	28.0	11.9	19.3	13.8
TA32	1774	42.7	33.3	19.0	22.1	18.2	TA67	2825	31.7	26.1	15.2	18.0	12.5
TA33	1788	51.3	34.8	27.5	25.4	18.2	TA68	2784	50.1	19.1	10.6	14.9	10.4
TA34	1828	45.9	31.8	20.7	21.0	15.8	TA69	3071	35.3	22.8	12.9	16.2	10.6
TA35	2007	30.3	24.1	8.4	10.8	5.1	TA70	2995	42.9	23.4	16.7	21.8	15.8

Statements and Declarations

Funding.

This work was supported by the Spanish Ministry of Science, Innovation and Universities (MCIN/AEI/10.13039/501100011033) under grant PID2022-141746OB-I00.

Competing interests.

The author has no competing interests to declare that are relevant to the content of this article.

Data availability.

The benchmark instances are openly available in a Zenodo repository (Díaz Rodríguez, 2026a). Code, benchmark harness, all experiment records (accepted and rejected), final checkpoints and figure-generation scripts will be deposited openly upon submission. [TODO: DOI del depósito propio al enviar]

References

Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S. (2016). *Neural combinatorial optimization with reinforcement learning.* (arXiv:1611.09940)

Branke, J., Nguyen, S., Pickardt, C.W., Zhang, M. (2016). Automated design of production scheduling heuristics: a review. *IEEE Transactions on Evolutionary Computation*, *20*(1), 110–124, https://doi.org/10.1109/tevc.2015.2429314

Díaz, H., González-Rodríguez, I., Palacios, J.J., Díaz, I., Vela, C.R. (2020). A genetic approach to the job shop scheduling problem with interval uncertainty. *Information processing and management of uncertainty in knowledge-based systems (IPMU)* (pp. 663–676). Springer.

Díaz, H., Palacios, J.J., Díaz, I., Vela, C.R., González-Rodríguez, I. (2022). Robust schedules for tardiness optimization in job shop with interval uncertainty. *Logic Journal of the IGPL*, *31*(2), 240–254, https://doi.org/10.1093/jigpal/jzac016

Díaz, H., Palacios, J.J., González-Rodríguez, I., Vela, C.R. (2023a). An elitist seasonal artificial bee colony algorithm for the interval job shop. *Integrated Computer-Aided Engineering*, *30*(3), 223–242, https://doi.org/10.3233/ica-230705

Díaz, H., Palacios, J.J., González-Rodríguez, I., Vela, C.R. (2023b). Fast elitist ABC for makespan optimisation in interval JSP. *Natural Computing*, *22*(4), 645–657, https://doi.org/10.1007/s11047-023-09953-2

Díaz Rodríguez, H. (2026a). *Genetic programming dispatching rules for the interval job shop: instances, evolved rules, results and code.* Zenodo. ([dataset])

Díaz Rodríguez, H. (2026b). *Genetic programming hyper-heuristics for the job shop scheduling problem with interval durations: robustness-aware interpretable rules that generalize across instance sizes.* (Preprint. [TODO: identificador del preprint])

Garey, M.R., Johnson, D.S., Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, *1*(2), 117–129, https://doi.org/10.1287/moor.1.2.117

Giffler, B., & Thompson, G.L. (1960). Algorithms for solving production-scheduling problems. *Operations Research*, *8*(4), 487–503, https://doi.org/10.1287/opre.8.4.487

González-Rodríguez, I., Vela, C.R., Puente, J. (2010). A genetic solution based on lexicographical goal programming for a multiobjective job shop with uncertainty. *Journal of Intelligent Manufacturing*, *21*(1), 65–73, https://doi.org/10.1007/s10845-008-0161-x

Haupt, R. (1989). A survey of priority rule-based scheduling. *OR Spektrum*, *11*(1), 3–16, https://doi.org/10.1007/bf01721162

Kool, W., van Hoof, H., Welling, M. (2019). Attention, learn to solve routing problems! *International conference on learning representations (ICLR).*

Lei, D. (2011). Population-based neighborhood search for job shop scheduling with interval processing time. *Computers & Industrial Engineering*, *61*(4), 1200–1208, https://doi.org/10.1016/j.cie.2011.07.010

Leon, V.J., Wu, S.D., Storer, R.H. (1994). Robustness measures and robust scheduling for job shops. *IIE Transactions*, *26*(5), 32–43, https://doi.org/10.1080/07408179408966626

López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Birattari, M., Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, *3*, 43–58, https://doi.org/10.1016/j.orp.2016.09.002

Nazari, M., Oroojlooy, A., Snyder, L., Takáč, M. (2018). Reinforcement learning for solving the vehicle routing problem. *Advances in neural information processing systems (NeurIPS).*

Nguyen, S., Mei, Y., Zhang, M. (2017). Genetic programming for production scheduling: a survey with a unified framework. *Complex & Intelligent Systems*, *3*(1), 41–66, https://doi.org/10.1007/s40747-017-0036-x

Palacios, J.J., González-Rodríguez, I., Vela, C.R., Puente, J. (2015). Coevolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop. *Fuzzy Sets and Systems*, *278*, 81–97, https://doi.org/10.1016/j.fss.2014.12.003

Panwalkar, S.S., & Iskander, W. (1977). A survey of scheduling rules. *Operations Research*, *25*(1), 45–61, https://doi.org/10.1287/opre.25.1.45

Park, J., Chun, J., Kim, S.H., Kim, Y., Park, J. (2021). Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research*, *59*(11), 3360–3377, https://doi.org/10.1080/00207543.2020.1870013

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. (2017). *Proximal policy optimization algorithms.* (arXiv:1707.06347)

Sotskov, Y.N., Matsveichuk, N.M., Hatsura, V.D. (2020). Schedule execution for two-machine job-shop to minimize makespan with uncertain processing times. *Mathematics*, *8*(8), 1314, https://doi.org/10.3390/math8081314

Taillard, E. (1993). Benchmarks for basic scheduling problems. *European Journal of Operational Research*, *64*(2), 278–285, https://doi.org/10.1016/0377-2217(93)90182-M

Tassel, P., Gebser, M., Schekotihin, K. (2021). *A reinforcement learning environment for job-shop scheduling.* (arXiv:2104.03760)

Đurašević, M., & Jakobović, D. (2018). A survey of dispatching rules for the dynamic unrelated machines environment. *Expert Systems with Applications*, *113*, 555–569, https://doi.org/10.1016/j.eswa.2018.06.053

Vinyals, O., Fortunato, M., Jaitly, N. (2015). Pointer networks. *Advances in neural information processing systems (NeurIPS).*

Xu, M., Mei, Y., Zhang, F., Zhang, M. (2025). Learn to optimise for job shop scheduling: a survey with comparison between genetic programming and reinforcement learning. *Artificial Intelligence Review*, *58*, 160, https://doi.org/10.1007/s10462-024-11059-9

Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R., Smola, A. (2017). Deep sets. *Advances in neural information processing systems (NeurIPS).*

Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P.S., Xu, C. (2020). Learning to dispatch for job shop scheduling via deep reinforcement learning. *Advances in neural information processing systems (NeurIPS).*

Zheng, P., Xiao, S., Zhang, P., Lv, Y. (2024). A two-individual-based evolutionary algorithm for flexible assembly job shop scheduling problem with uncertain interval processing times. *Applied Sciences*, *14*(22), 10304, https://doi.org/10.3390/app142210304